Supermarket POS System

1. Course Name

Cap1: System Analysis and Design

2. Team Information

Team Name: Prime Logic Inc.

Name	Group	Student ID	Role
Bobur	I24A	202490377	Leader
Azizjon	I24A	202490017	Project Manager
Ulug'bek	I24A	202490206	Designer
Erkin	I24A	202490286	Software Engineer
Kamalatdin	I24A	202490284	Software Engineer
Nusrat	I24B	202490199	Software Engineer

3. Project Title / Description

This project aims to develop a database-driven **Supermarket POS System** using SQL. The system records and manages payment information when a customer purchases a product through a Point of Sale (POS).

It stores key details such as:

- Customer identity
- Purchased products and quantity
- Total amount
- Payment method
- POS location
- Transaction date and time

Each transaction is first recorded with a Pending status and later updated to Confirmed or Failed based on the payment result.

The database uses multiple related tables:

- Customers

- Products
- POS Terminals
- Transactions
- Transaction Items

This system demonstrates how businesses track sales, confirm payments, and maintain reliable transaction records for reporting and analysis.

4. Problem Statement

Businesses need a reliable way to record and verify customer payments made through POS systems. Without a structured system, transaction data may be lost, inconsistent, or difficult to track, causing errors in payment confirmation and financial reporting.

This project solves the problem by creating a structured SQL database that:

- Securely stores transaction details
- Tracks payment status (Pending, Confirmed, Failed)
- Maintains accurate records of who purchased what, when, and where

5. Project Objectives

- Design a structured SQL database for transaction management
- Record customer, product, POS, and time information
- Implement payment status tracking
- Ensure data integrity using relationships and constraints
- Enable easy retrieval of transaction data for monitoring and reporting

6. Scope of the System

The system manages transaction data generated from a POS. It includes:

- Customer information
- Product details
- POS terminal data
- Transaction records with items, amount, payment method, status, and timestamp

Limitations:

- No user interface
- No real payment gateway integration
- No inventory management or authentication
- Designed for educational purposes only

7. Expected Users

- Store Cashiers – record sales transactions
- Store Managers – review sales and transaction history
- Business Owners – analyze revenue and performance

- System Administrators – maintain the database

8. Project Timeline (5 Weeks)

Week	Activities
Week 1 (06 Feb – 13 Feb)	Project planning
Week 2 (13 Feb – 20 Feb)	BPM, Use Case
Week 3 (20 Feb – 27 Feb)	BPM, Use Case
Week 4 (27 Feb – 06 Mar)	ERD Design
Week 5 (06 Mar – 13 Mar)	Report writing & presentation

9. Project Outcomes

- Understand relational database design
- Gain practical SQL experience (CREATE, INSERT, UPDATE, SELECT)
- Learn transaction management and table relationships
- Understand real-world payment and sales processes
- Improve problem-solving and system design skills

10. Conclusion

The project provided a comprehensive introduction to relational database management, beginning with the critical technical task of local SQL environment configuration. By leveraging a collaborative "vibe coding" methodology and integrating AI-driven code generation, the development process was streamlined, allowing for a primary focus on high-level logic and system architecture rather than manual syntax.

Furthermore, proficiency was established in utilizing the **draw.io** (app.diagrams.net) web interface for technical documentation. This involved the formal translation of system requirements into standardized visual models, specifically **Use Case Diagrams**, **Business Process Models (BPM)**, and **Entity-Relationship Diagrams (ERD)**. The synthesis of these tools facilitated a robust understanding of both backend implementation and structured system design.

11. References

Tutorials to use draw.io

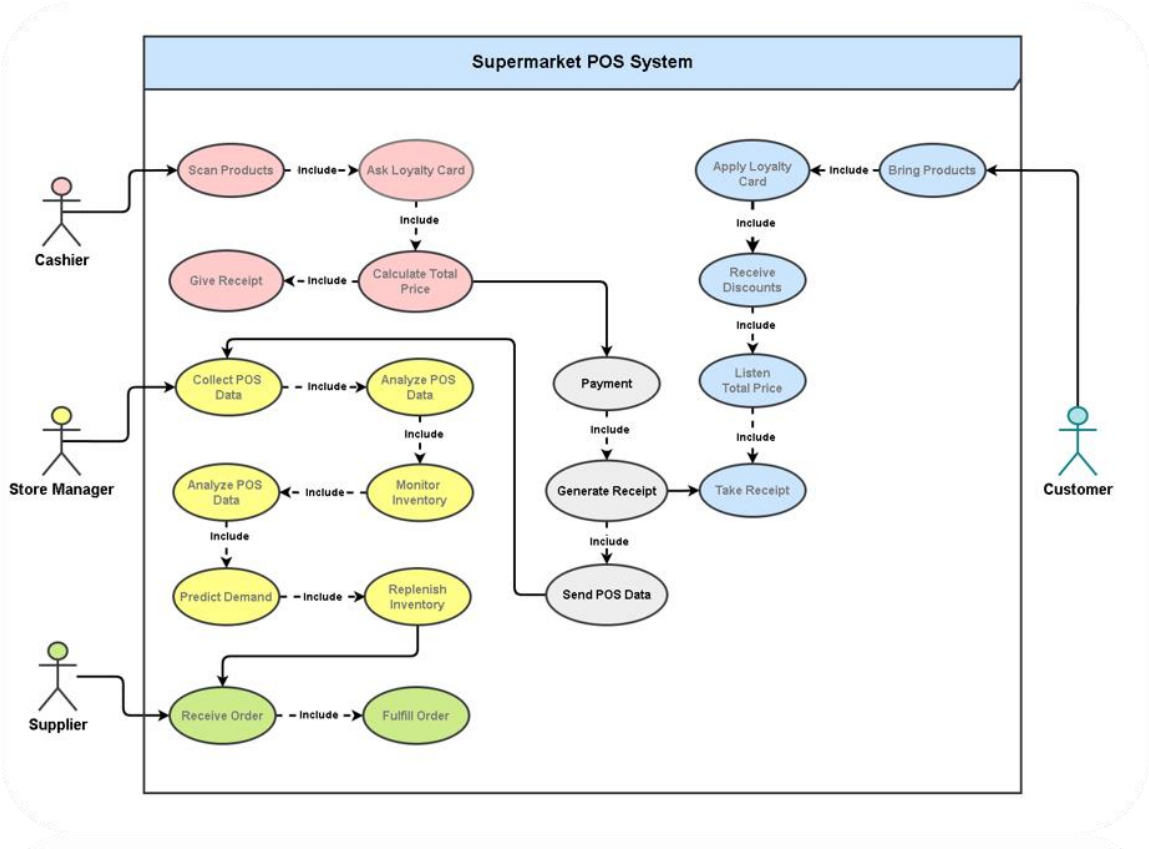
https://youtu.be/1T4igDVMVEg?si=j7No6fdE6kwL_tx4

https://youtu.be/_MzTijsoSe8?si=b-XU5kWIGHU4IU1-

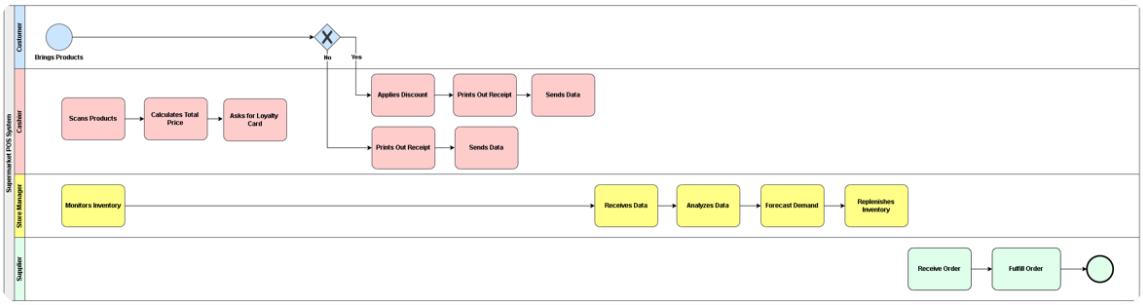
https://youtu.be/JYZPdU5F2iM?si=rdwDp-VCgoB28a1b

Tutorial to install MySQL: https://youtu.be/hiS_mWZmmI0?si=1NjFijY2WsoAqJvd

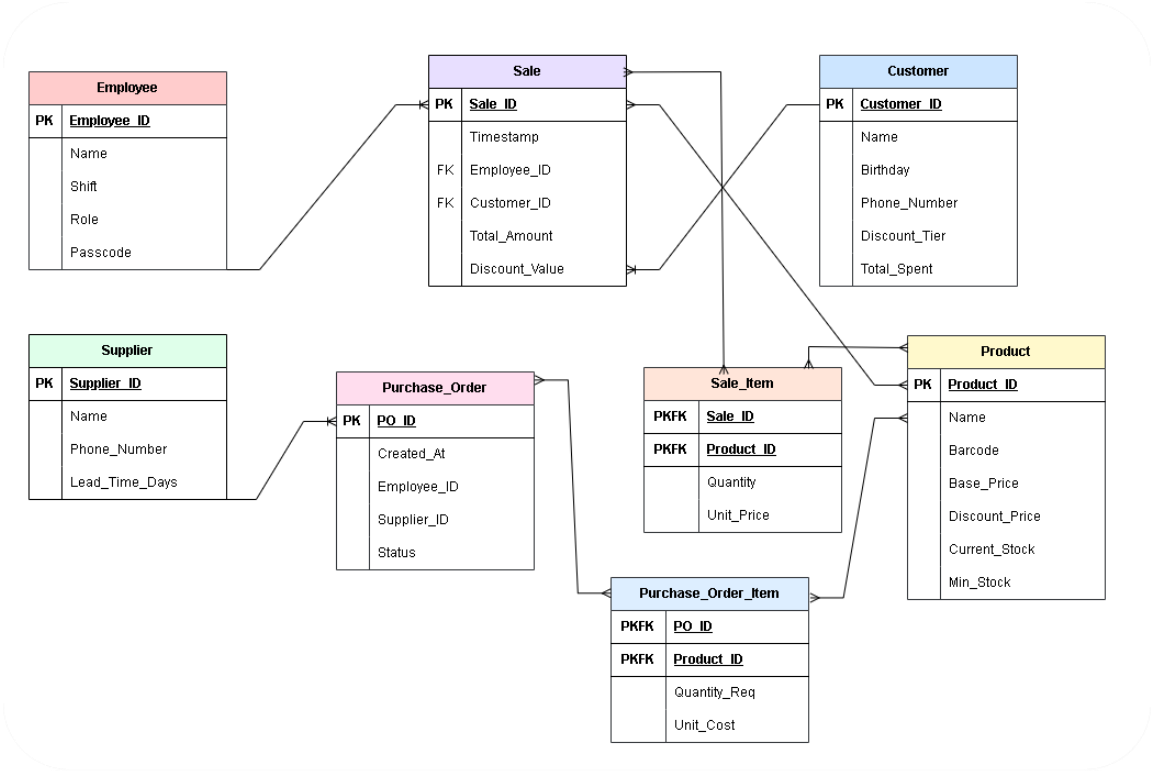
12. Appendix



Use Case



Business Process Model



Entity Relationship Diagram

SQL Code:

```
-- ========================================
-- CREATE DATABASE
-- ========================================

CREATE DATABASE Prime_Logic;

USE Prime_Logic;

-- ========================================
-- 1. INDEPENDENT TABLES
-- ========================================

CREATE TABLE Employee (
    Employee_ID INT AUTO_INCREMENT PRIMARY KEY,
    First_Name VARCHAR(50) NOT NULL,
    Last_Name VARCHAR(50) NOT NULL,
    Role ENUM('Cashier','Manager','Admin') NOT NULL,
    Passcode_Hash VARCHAR(255) NOT NULL
);
```

```
CREATE TABLE Customer (
    Customer_ID INT AUTO_INCREMENT PRIMARY KEY,
    Phone_Number VARCHAR(15) UNIQUE,
    Discount_Tier DECIMAL(3,2) NOT NULL,
    Total_Spent DECIMAL(10,2) DEFAULT 0.00
);

CREATE TABLE Supplier (
    Supplier_ID INT AUTO_INCREMENT PRIMARY KEY,
    Company_Name VARCHAR(100) NOT NULL,
    Contact_Email VARCHAR(100) NOT NULL,
    Lead_Time_Days INT NOT NULL CHECK (Lead_Time_Days >= 0)
);

CREATE TABLE Product (
    Product_ID INT AUTO_INCREMENT PRIMARY KEY,
    Barcode VARCHAR(50) UNIQUE NOT NULL,
    Product_Name VARCHAR(100) NOT NULL,
    Base_Price DECIMAL(10,2) NOT NULL CHECK (Base_Price >= 0),
    Current_Stock INT NOT NULL CHECK (Current_Stock >= 0),
    Min_Threshold INT NOT NULL CHECK (Min_Threshold >= 0)
);

-- ==========================================
-- 2. DEPENDENT TABLES
-- ==========================================

CREATE TABLE Sale (
    Sale_ID INT AUTO_INCREMENT PRIMARY KEY,
    Timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    Employee_ID INT NOT NULL,
    Customer_ID INT,
    Total_Amount DECIMAL(10,2) NOT NULL CHECK (Total_Amount >=
0),
    Discount_Value DECIMAL(10,2) DEFAULT 0.00,

    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,

    FOREIGN KEY (Customer_ID)
        REFERENCES Customer(Customer_ID)
        ON DELETE SET NULL
        ON UPDATE CASCADE
```

```
);

CREATE TABLE Purchase_Order (
    PO_ID INT AUTO_INCREMENT PRIMARY KEY,
    Created_At DATETIME DEFAULT CURRENT_TIMESTAMP,
    Employee_ID INT NOT NULL,
    Supplier_ID INT NOT NULL,
    Status ENUM('Pending','Ordered','Received','Cancelled')
NOT NULL,

    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,

    FOREIGN KEY (Supplier_ID)
        REFERENCES Supplier(Supplier_ID)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

-- ========================================
-- 3. ASSOCIATIVE TABLES
-- ========================================

CREATE TABLE Sale_Item (
    Sale_ID INT,
    Product_ID INT,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    Unit_Price DECIMAL(10,2) NOT NULL CHECK (Unit_Price >= 0),

    PRIMARY KEY (Sale_ID, Product_ID),

    FOREIGN KEY (Sale_ID)
        REFERENCES Sale(Sale_ID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    FOREIGN KEY (Product_ID)
        REFERENCES Product(Product_ID)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE Purchase_Order_Item (
```

```
    PO_ID INT,
    Product_ID INT,
    Quantity_Req INT NOT NULL CHECK (Quantity_Req > 0),
    Unit_Cost DECIMAL(10,2) NOT NULL CHECK (Unit_Cost >= 0),

    PRIMARY KEY (PO_ID, Product_ID),

    FOREIGN KEY (PO_ID)
        REFERENCES Purchase_Order(PO_ID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    FOREIGN KEY (Product_ID)
        REFERENCES Product(Product_ID)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```